# Smart Pothole Detection System

Graduation Project Committee Presentation

Computer Engineering - Yarmouk University

Team Members: Omar Awawdeh, Hamza Al-Safi

Academic Year: 2025-2026

Committee Presentation

# Problem Statement

Why a smart pothole monitoring workflow is needed

Manual road inspection is slow, costly, and difficult to keep up-to-date across large areas.

Potholes create safety risks, vehicle damage, and higher maintenance costs when they are reported late.

Citizens may report issues, but reports are often inconsistent, duplicated, or missing precise location data.

Road authorities need timely, GPS-tagged evidence to prioritize maintenance more effectively.

### Core Idea

Use a smartphone camera and on-device AI to detect potholes in real time, attach GPS coordinates,

and send structured reports to a central dashboard for verification and tracking.

### Why On Device?

Lower latency

Works with limited connectivity

Lower bandwidth and cloud cost

Better privacy than streaming video to the cloud

# Objectives

Target outcomes for the complete system

### Detect

Identify potholes from live camera frames in real time on Android.

### Locate

Capture GPS coordinates for each valid detection.

### Report

Upload structured reports with images and metadata to the backend.

### Visualize

Provide a dashboard with map, records, and status management.

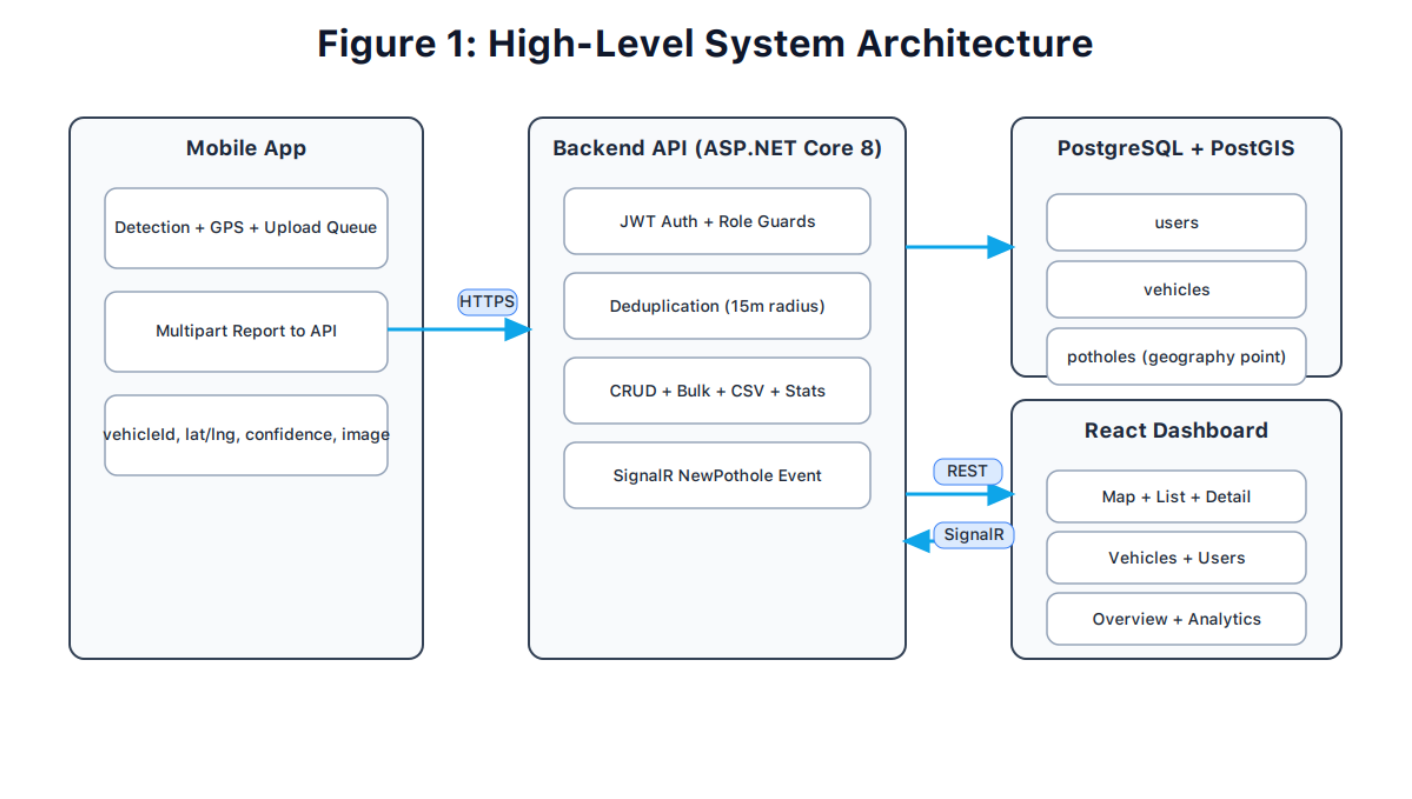
### Deploy

Run the full system on a VPS with a production-style architecture.

**This project is designed as an end-to-end workflow, not only an isolated AI model.**

# Proposed Solution

End-to-end workflow from detection to maintenance visibility




Android app captures live frames and runs YOLOv8n through TensorFlow Lite.

Valid detections are filtered, geotagged, deduplicated locally, and queued for upload.

Backend stores pothole records in PostgreSQL/PostGIS and manages report lifecycle.

Dashboard allows authorities to review locations, images, and repair status.

# System Architecture

Current implementation stack in the repository

### Mobile Layer

Kotlin

Jetpack Compose

CameraX

TensorFlow Lite

GPS + WorkManager

### AI Layer

YOLOv8n object detector

Float16 TFLite export

Recall-tuned thresholds

### Backend Layer

ASP.NET Core 8 Web API

Entity Framework Core

REST endpoints

### Data Layer

PostgreSQL 16

PostGIS spatial support

Cloudflare R2 image storage

### Dashboard + Ops

React + Vite + Tailwind

React Leaflet map

Docker Compose + NGINX on VPS

# AI Model and Dataset

Why YOLOv8n is suitable for mobile pothole detection

Model: YOLOv8n (nano), selected for a strong speed/accuracy trade-off on mobile devices.

Dataset: 2,642 images and 9,077 labeled potholes collected from public datasets.

Data split: 80.5% train, 14.5% validation, 5.0% test.

Deployment format: float16 TFLite model, about 6 MB, suitable for Android packaging.

**2,642**
Images

**9,077**
Annotations

**~6 MB**
TFLite Size

**25-30 FPS**
Expected Mobile Speed

# AI Results Charts

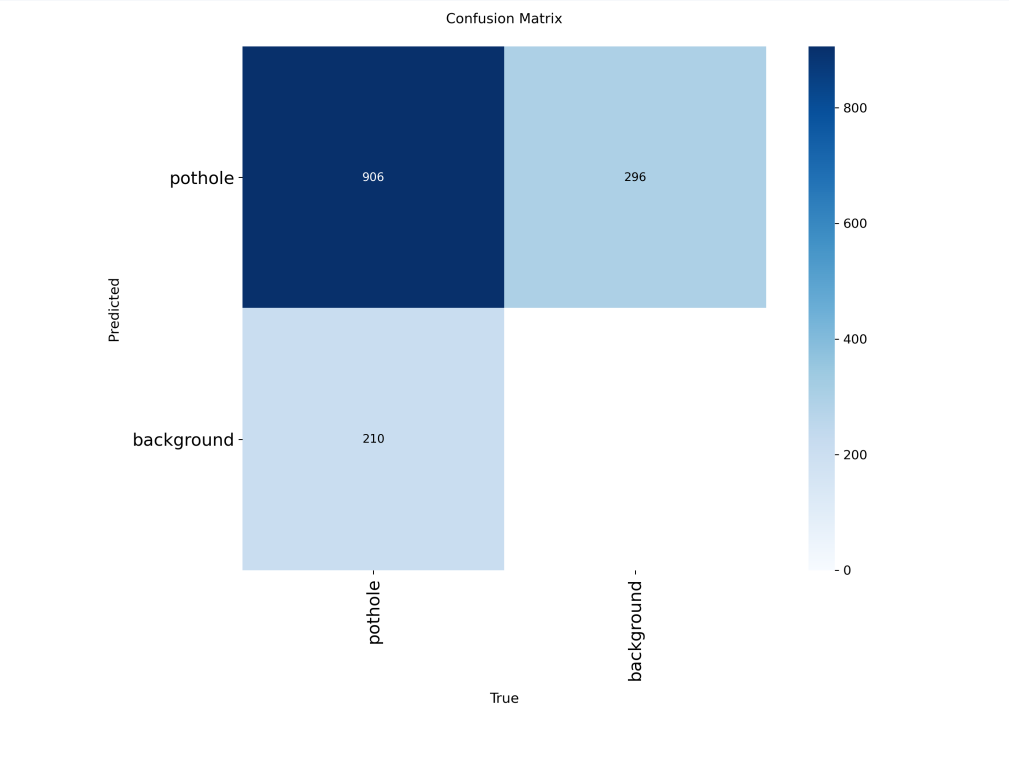
Visual evidence from model evaluation and training history

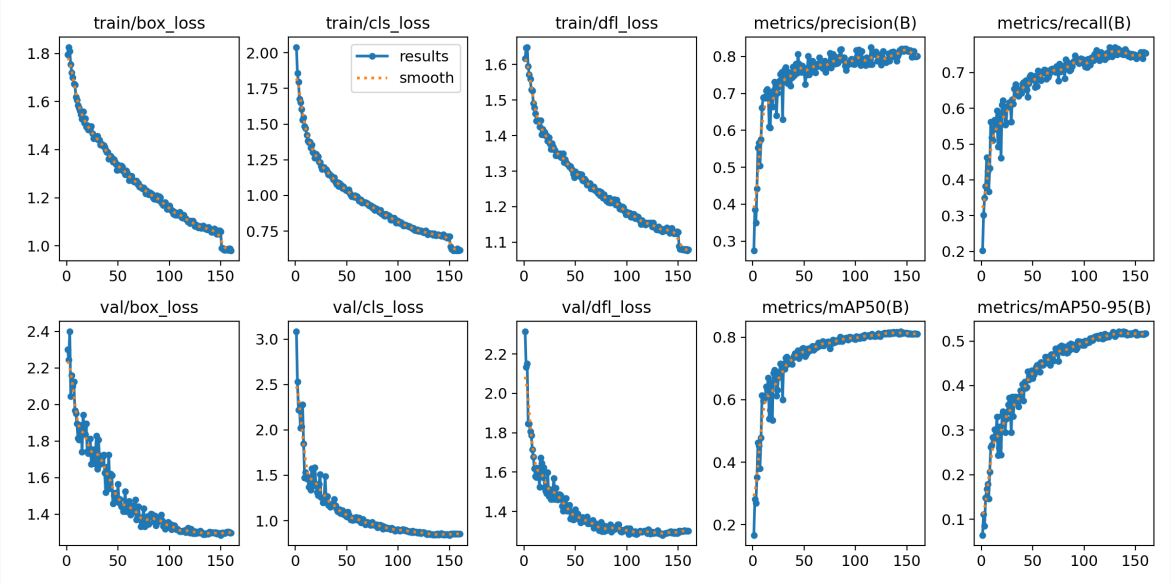
## Confusion Matrix

Shows how well the model separates pothole detection from background and errors.

## Training Curves

Shows loss reduction and metric improvement during training, supporting model convergence.



# AI Results

Best validation metrics and deployment operating point

| 81.45% | 52.13% | 79.98% | 75.90% |
|---|---|---|---|
| mAP@50 | mAP@50-95 | Precision | Recall |

**Interpretation**

The tuned model keeps precision near 80% while reducing missed potholes.
This trade-off is suitable for a field reporting system where missing defects is costly.

# Android Application

Main features implemented in the mobile app

Live camera preview with real-time pothole detection overlay.

On-device inference using TensorFlow Lite for low-latency performance.

GPS integration to tag each valid detection with location metadata.

History and settings screens for review and configuration.

Background upload queue for reliable synchronization when connectivity changes.

**Engineering Focus**

Low latency

Offline-friendly flow

Mobile-first UX

Reliable upload behavior

**Representative Screens**

Detection

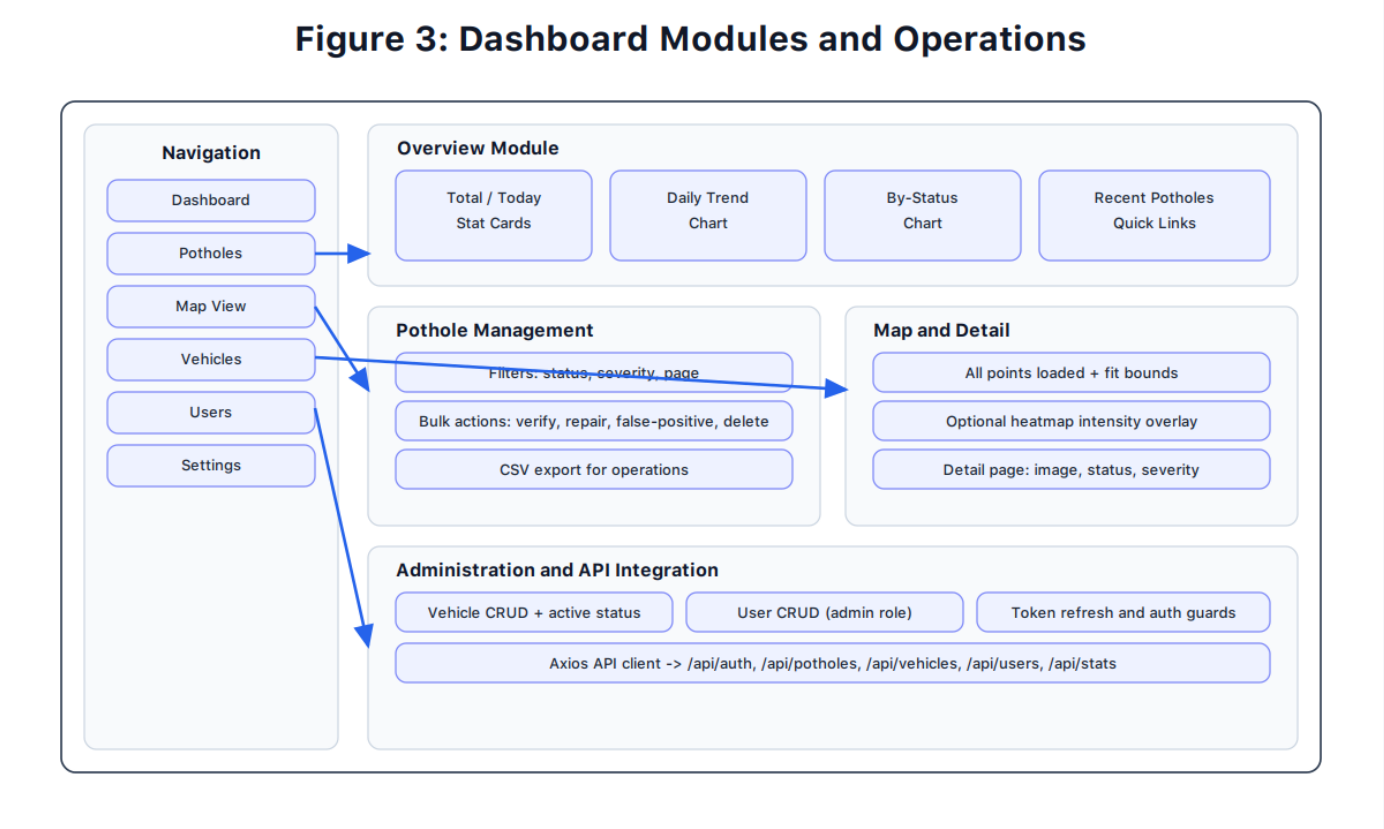
History

Settings

Debug / verification support

# Backend and Dashboard

How reports are stored, reviewed, and managed



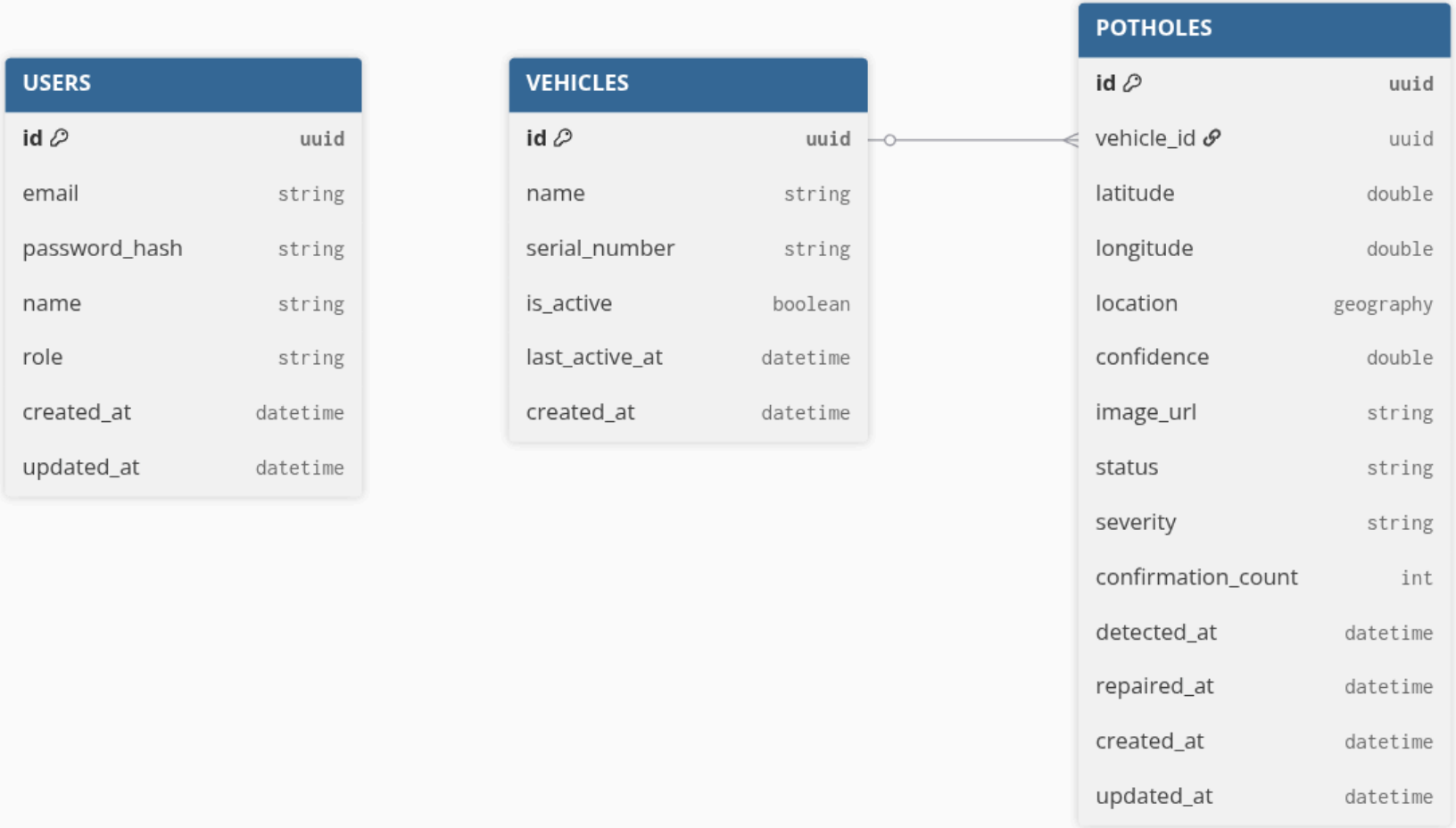

Backend uses ASP.NET Core 8 with PostgreSQL/PostGIS for API and spatial data handling.

Dashboard uses React, Vite, Tailwind, and React Leaflet for map-based monitoring.

Authorities can browse pothole records, inspect images, and update status such as verified or repaired.

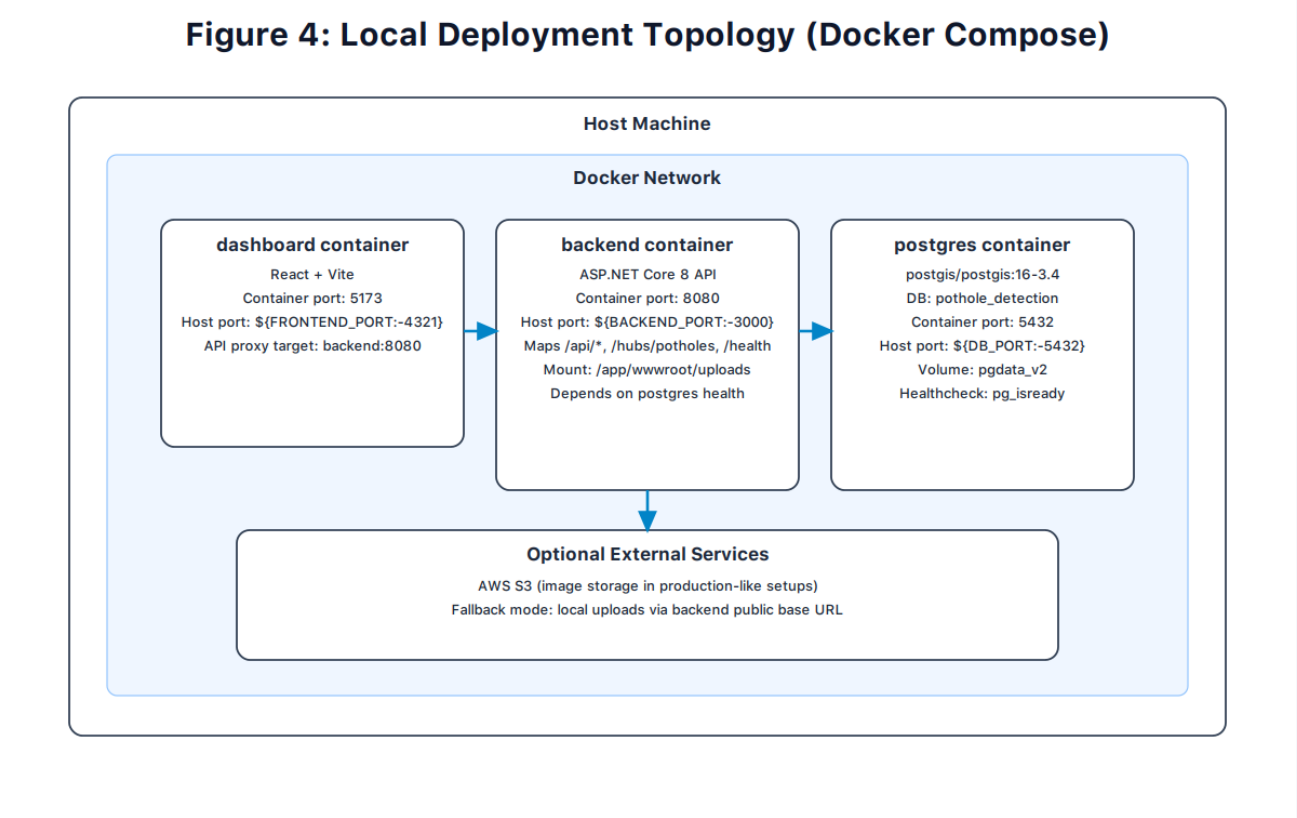
Map view loads all pothole points and auto-fits bounds for easier monitoring.

# Database Design



# Deployment and Demo Readiness

Production-style setup used for the integrated system




System is deployed on a Linux VPS using Docker Compose and NGINX.

Dashboard domain: potholesystem.tech; API domain: api.potholesystem.tech.

PostgreSQL + PostGIS provides persistent geospatial storage.

Uploaded images can be served from Cloudflare R2 through a public image domain.

# Limitations and Future Work

What can be improved after the current prototype

### Current Limitations

Recall is improved but some potholes are still missed.

Training data is mostly public and may not fully match local Jordan roads.

Performance can vary with lighting, glare, weather, and motion blur.

Current model detects potholes only, without severity grading.

### Future Work

Collect local road data and fine-tune on regional conditions.

Add severity classification and other road-hazard classes.

Validate the tuned TFLite export more extensively on target devices.

Expand robustness for night-time and adverse weather conditions.

# Conclusion

An end-to-end AI-assisted road monitoring solution

The project delivers a complete workflow: detection, geotagging, reporting, storage, and dashboard monitoring.

The mobile AI model achieves acceptable results while remaining deployable on Android devices.

The integrated architecture demonstrates practical value for municipalities and road maintenance teams.

This work can be extended into a larger smart-city road condition monitoring platform.

## Thank You

Questions and Discussion